Numerical Simulation Platform for the Non-Markovian to Markovian Transition in Open Quantum System Dynamics

PH3406 Open Quantum Systems – Technical Project Report

Tisorn Na Phattalung, U2420781G *

Abstract

This report describes a modular Python simulation platform built to numerically reproduce and extend the four experiments of Gaikwad *et al.* [1], which probe the non-Markovian (NM) to Markovian transition in open quantum system dynamics using three superconducting transmon qubits. Starting from the full system Hamiltonian, the platform implements two levels of approximation: the interaction-picture/RWA (level 0), which exactly reproduces the paper's results, and a doubly-rotating frame with AC Stark corrections (level 1). All four experiments are replicated qualitatively and, for Experiments 1 and 4, quantitatively. Six creative extensions are included: Monte Carlo quantum trajectories, Bloch–Redfield structured-bath modelling, hardware-faithful maximum-likelihood quantum state tomography (QST), quantum process tomography (QPT), a custom PennyLane circuit runner, and a standalone parametric modulation module demonstrating the Jacobi–Anger sideband selection mechanism. The report concludes with a critical reflection on the role of the AI agent in the project.

Keywords: Non-Markovian dynamics, Superconducting qubits, GKSL master equation, Concurrence, Quantum Zeno effect, QuTiP, Parametric modulation

Contents

1 Introduction **2**
1.1 Motivation . . . 2
1.2 Physical System: Superconducting Transmon Qubits . . . 2
1.3 Full System Hamiltonian . . . 3
1.4 Experimental Programme . . . 3

2 Theoretical Framework **3**
2.1 From Nakajima–Zwanzig to GKSL . . . 3
2.2 Concurrence and Non-Markovianity Measure . . . 3
2.3 Quantum Zeno Effect . . . 4

3 Simulation Platform **4**
3.1 Architecture Overview . . . 4
3.2 Approximation Levels . . . 4
3.3 Operator Embedding and Hamiltonians . . . 5
3.4 Collapse Operators . . . 5
3.5 Bell-State Preparation . . . 5

4 Experiment Replication **6**
4.1 Experiment 1 – Markovian Baseline . . . 6
4.2 Experiment 2 – Non-Markovian Dynamics . . . 6
4.3 Experiment 3 – NM to Markovian Transition . . . 6
4.4 Experiment 4 – Quantum Zeno Stabilisation . . . 6

5 Creative Extensions **6**
5.1 Monte Carlo Quantum Trajectories . . . 6
5.2 Bloch–Redfield Structured-Bath Modelling . . . 7
5.3 Hardware-Faithful Quantum State Tomography . . . 7
5.4 Quantum Process Tomography . . . 7

*U2420781G, School of Electrical and Electronic Engineering, Nanyang Technological University. Source code: `github.com/tisorn25484911/ Superconducting-QubitMarkov-NonMarkov-Transition-Simulation`

5.5 Custom PennyLane Circuit Runner . . . 7
5.6 Parametric Modulation Module (`modulation.py`) . . . 7

6 AI Reflection Piece **7**
6.1 Division of Labour . . . 7
6.2 Successes of AI-Assisted Structuring . . . 7
6.3 Failures and Limitations . . . 8
6.4 Assessment of Academic Integrity . . . 8

7 Summary **8**

Appendix A Code Pipeline and Experiment Replication Functions **8**
Appendix A.1 `state_prep.py` – Operators, Hamiltonians, States . . . 9
Appendix A.1.1 Operator embedding . . . 9
Appendix A.1.2 Hamiltonian hierarchy . . . 9
Appendix A.1.3 Bell-state preparation . . . 9
Appendix A.1.4 Non-Markovianity measure and Wootters concurrence . . . 9
Appendix A.2 `dynamical.py` – Collapse Operators and Primary Solver . . . 10
Appendix A.2.1 Collapse operator builder . . . 10
Appendix A.2.2 Primary Lindblad solver . . . 10
Appendix A.2.3 Zeno scaling utilities . . . 10
Appendix A.3 `experiment.py` – Experiment Protocols . . . 10
Appendix A.3.1 `exp1()` – Markovian baseline . . . 10
Appendix A.3.2 `exp2()` – Non-Markovian dynamics . . . 11
Appendix A.3.3 `exp3()` – NM to Markovian transition . . . 11
Appendix A.3.4 `exp4()` – Quantum Zeno stabilisation . . . 11
Appendix A.4 `main.py` – Entry-Point Pipeline . . . 11

Appendix B Creative Extension Functions **11**
Appendix B.1 Extension 1 – Monte Carlo Quantum Trajectories . . . 11
Appendix B.2 Extension 2 – Bloch–Redfield Structured-Bath Modelling . . . 12
Appendix B.3 Extension 3 – Hardware-Faithful Quantum State Tomography . . . 12
Appendix B.4 Extension 4 – Maximum-Likelihood Quantum State Tomography . . . 13
Appendix B.5 Extension 5 – Quantum Process Tomography . . . 13
Appendix B.6 Extension 6 – Custom PennyLane Circuit Runner . . . 13
Appendix B.7 Extension 7 – Parametric Modulation Module . . . 14

1. Introduction

1.1. Motivation

Decoherence – the loss of quantum coherence through entanglement with environmental degrees of freedom – is the principal obstacle to large-scale quantum computation. The standard description, the Gorini–Kossakowski–Sudarshan–Lindblad (GKSL) master equation, rests on two approximations: *weak coupling* and a *memoryless* environment whose correlation time satisfies $\tau_E \ll \tau_S$. When these fail – as with small, structured, or strongly coupled environments – non-Markovian dynamics emerge: information flows *back* from the environment, producing effects the GKSL framework cannot capture, most directly an *increase* of entanglement over time.

Gaikwad *et al.* [1] address this using a superconducting transmon processor: they couple a system qubit to a controllable quantum environment, use entanglement revival as an unambiguous witness of quantum memory, and then tune a dissipation parameter to drive the system continuously through the NM $\rightarrow$ Markovian transition, ultimately reaching a quantum Zeno regime where more dissipation paradoxically *stabilises* entanglement. This project numerically reproduces and extends these experiments.

1.2. Physical System: Superconducting Transmon Qubits

The device consists of three transmon qubits fabricated at MIT Lincoln Laboratory, operated at millikelvin temperatures in the 4–5 GHz range. A transmon is an anharmonic LC oscillator formed by shunting a Josephson junction with a large capacitor. Its Hamiltonian is

$$H_{\text{transmon}} = 4E_C(\hat{n} - n_0)^2 - E_J \cos\left(\frac{2\pi\Phi}{\Phi_0}\right), \tag{1}$$

where E_C and E_J are the charging and Josephson energies, $\hat{n}$ is the Cooper-pair number operator, and Φ/Φ_0 is the reduced external flux. The Josephson nonlinearity breaks the equal level spacing of a harmonic oscillator, enabling selective two-level addressing of $|g\rangle$ and $|e\rangle$ via the transition frequency ω_{01}.

Crucially, $\omega_{01}(\Phi)$ is tunable via an on-chip fast flux line (FFL), which is the mechanism exploited for parametric coupling.

Table 1: Role assignment and key parameters of the three transmon qubits [1].

Qubit	Role and properties
Qubit (Q)	Frequency-tunable (SQUID + FFL); system qubit of interest. Parametrically coupled to both A and E. $T_2^{*(Q)} = 39\,\mu$s, $T_1^{(Q)} = 31\,\mu$s.
Ancilla (A)	Fixed-frequency entanglement probe; Bell partner of Q. $T_2^{*(A)} = 41\,\mu$s, $T_1^{(A)} = 32\,\mu$s.
Environment (E)	Frequency-tunable quantum bath; dispersively coupled to a microwave readout resonator. Thermal photons injected into the resonator tune the dephasing rate γ of E. $T_2^{*(E)} = 38\,\mu$s, $T_1^{(E)} = 28\,\mu$s.

1.3. Full System Hamiltonian

The three-qubit Hamiltonian in the lab frame is

$$H = \frac{\hbar\omega_E}{2}\hat{Z}_E + \frac{\hbar\omega_Q}{2}\hat{Z}_Q + \frac{\hbar\omega_A}{2}\hat{Z}_A + \frac{\Omega_{QA}}{2}\Big(\sigma_+^Q\sigma_-^A + \text{h.c.}\Big) + \frac{\Omega_{QE}}{2}\Big(\sigma_+^Q\sigma_-^E + \text{h.c.}\Big) + \chi\, a^\dagger a\, \hat{Z}_E, \tag{2}$$

where the iSWAP couplings $\Omega_{QA}/2\pi = 0.477\,\text{MHz}$ and $\Omega_{QE}/2\pi = 0.473\,\text{MHz}$ are activated by parametric modulation of the FFL at $f_{\text{mod}} = \Delta_{Q,A}/2$ (QA coupling, via the Jacobi–Anger n=2 sideband) and $\Delta_{Q,E}/4$ (QE coupling, n=4 sideband with anti-phase E drive), respectively. The last term $\chi a^\dagger a\,\hat{Z}_E$ ($\chi/2\pi = 200\,\text{kHz}$) is the dispersive interaction between E and its readout resonator; thermal photons injected here produce Markovian dephasing at rate $\gamma = 1.84\ (\mu\text{s})^{-1} A_{\text{in}}^{1.5}$.

1.4. Experimental Programme

The four experiments probe distinct aspects of the open-system dynamics, summarised in Table 2.

2. Theoretical Framework

2.1. From Nakajima–Zwanzig to GKSL

Tracing the total von-Neumann equation over environment degrees of freedom gives the exact Nakajima–Zwanzig equation:

$$\frac{d\rho_S}{dt} = -\frac{i}{\hbar}[H_S, \rho_S] - \int_0^t \mathcal{K}(t-\tau)\,\rho_S(\tau)\,d\tau, \tag{3}$$

where $\mathcal{K}$ is the memory kernel encoding all system–environment correlations. The **Born approximation** ($\rho_{\text{tot}} \approx \rho_S \otimes \rho_E$, weak coupling) removes correlations and fixes the Lindblad rates to constants $\gamma_k \in \mathbb{R}^+$. The **Markov approximation** ($\tau_E \ll \tau_S$) eliminates the convolution, yielding the time-local GKSL master equation:

$$\boxed{\frac{d\rho_S}{dt} = -\frac{i}{\hbar}[H_S, \rho_S] + \sum_k \gamma_k \Big(L_k \rho_S L_k^\dagger - \tfrac{1}{2}\{L_k^\dagger L_k, \rho_S\}\Big)\,.} \tag{4}$$

Solutions of (4) are contractive: concurrence can only decrease. Any increase unambiguously proves non-Markovianity.

2.2. Concurrence and Non-Markovianity Measure

The Wootters concurrence of the reduced Q–A state is

$$\mathcal{C}(\rho_{QA}) = \max\!\big(0, \sqrt{\lambda_1} - \sqrt{\lambda_2} - \sqrt{\lambda_3} - \sqrt{\lambda_4}\big), \tag{5}$$

where λ_i are eigenvalues of $R = \rho_{QA}(\sigma_y \otimes \sigma_y)\rho_{QA}^*(\sigma_y \otimes \sigma_y)$ in descending order. The Rivas–Huelga–Plenio measure isolates the total entanglement recovered during all revival events:

$$\mathcal{N} = \int_{t_0}^{t_f} \left|\frac{d\mathcal{C}}{dt}\right| dt \;-\; \Delta\mathcal{C}, \quad \Delta\mathcal{C} = \mathcal{C}(t_0) - \mathcal{C}(t_f). \tag{6}$$

Table 2: Summary of the four experimental protocols of Gaikwad *et al.* [1].

#	Name	Configuration	Key observable	Expected result
1	**Bell-state free decay**	Q–A entangled via $\sqrt{\text{iSWAP}}$; QE coupling **off**; no thermal noise.	$\mathcal{C}(t)$ vs time.	Monotonic exponential decay: $\mathcal{C} = e^{-t/T_2^{*(Q)} - t/T_2^{*(A)}}$. $\mathcal{N} = 0$. Markovian GKSL baseline confirmed.
2	**Non-Markovian dynamics**	Q–A in Bell state; QE coupling **on** ($\Omega_{QE}/2\pi = 0.473\,\text{MHz}$); $\gamma = 0$ (no thermal noise on E).	$\mathcal{C}(t)$ and $\mathcal{N}$.	Concurrence collapses then **revives** at $t \sim 3$–$5\,\mu$s. $\mathcal{N} = 1.4 \gg 0$. Proves quantum memory in E.
3	**NM → Markovian transition**	Same as Exp. 2; sweep γ from 0 to $\sim 5\,\mu\text{s}^{-1}$ by injecting pseudo-thermal photons into E's resonator.	$\mathcal{N}(\gamma)$ and $\mathcal{C}(t, \gamma)$.	$\mathcal{N} \to 0$ continuously near $\gamma \simeq 1\,\mu\text{s}^{-1}$. Fully Markovian at $\gamma \gtrsim 3\,\mu\text{s}^{-1}$.
4	**Quantum Zeno stabilisation**	Deep Markovian regime: $\gamma \gg \Omega_{QE}$ (up to $\gamma = 45.5\,\mu\text{s}^{-1}$).	Effective decay rate Γ_c vs γ.	$\Gamma_c = \Omega_{QE}^2/(4\gamma) + \Gamma_0$. More dissipation on E **protects** Q–A entanglement; $\Gamma_c \to \Gamma_0$ as $\gamma \to \infty$.

2.3. Quantum Zeno Effect

Frequent measurements project E before Q–E superposition builds. For N measurements in interval T, the survival probability is

$$P_0(T) = \left[1 - \frac{(\Delta H)^2 T^2}{\hbar^2 N^2}\right]^N \xrightarrow{N\to\infty} 1. \tag{7}$$

With thermal-photon jump rate $\gamma \gg \Omega_{QE}$, the effective concurrence decay rate follows:

$$\Gamma_c = \frac{\Omega_{QE}^2}{4\gamma} + \Gamma_0, \qquad \Gamma_0 = \frac{1}{T_2^{*(Q)}} + \frac{1}{T_2^{*(A)}}. \tag{8}$$

As $\gamma \to \infty$, $\Gamma_c \to \Gamma_0$: E is effectively decoupled by its own dissipation.

3. Simulation Platform

3.1. Architecture Overview

The platform consists of five Python modules with a strict one-directional dependency chain. A single `DEFAULT_PARAMS` dictionary centralises all physical constants; a flag `params["aprox_deg"]` (0 or 1) propagates the chosen approximation level throughout every downstream calculation.

3.2. Approximation Levels

Why `H_lab_parametric` is excluded from the main pipeline. The Jacobi–Anger RWA requires the hierarchy $g_{\text{bare}} \ll \omega_m \ll \Delta$. With current parameters, $\Delta_{QA}/2\pi \approx 350\,\text{kHz}$ while $g_{\text{bare}}/2\pi \approx 980\,\text{kHz}$: the coupling exceeds the detuning, inverting the hierarchy and producing unphysical rapid Rabi oscillations ($\mathcal{N} \approx 15$) at level 1. `H_lab_parametric` is instead available in `modulation.py` as an educational demonstration; the main pipeline uses `H_for_simulation` (i.e. `H_QE_int + H_detuning`) for both levels.

Table 3: Module architecture and primary responsibilities.

File	Layer	Primary responsibilities
`state_prep.py`	Foundation	Operator embedding (`Q/A/E`), Hamiltonians (`H_free`, `H_QA_int`, `H_QE_int`, `H_detuning`, `H_for_simulation`, `H_lab_parametric`), frame transforms, Bell-state prep, `nm_measure`, `concurrence_manual`.
`dynamical.py`	Evolution & Measurement	`evolve_lindblad`, `evolve_born_markov`, `evolve_mcsolve`, `measure_state`, `full_tomography_shots`, `qst_log_likelihood`, `qpt_chi_matrix`, `build_c_ops`, `fit_decay_rate`, `gamma_c_zeno`.
`experiment.py`	Protocols	`exp1`–`exp4`, `plot_full_dashboard`, `run_custom_circuit`.
`modulation.py`	Lab-Frame Demo	`H_modulation_lab`, `scan_amplitude`, `plot_amplitude_scan` – standalone Jacobi–Anger demonstration.
`main.py`	Entry Point	Runs `exp1`–`exp4` sequentially; generates `simulation_dashboard.pdf`.

Table 4: The two approximation levels and their Hamiltonians.

Level	Hamiltonian	Physical interpretation
0 (default)	$H = H_{\mathrm{QE,int}} = \frac{\Omega_{QE}}{2}(\sigma_+^Q\sigma_-^E + \mathrm{h.c.})$	Interaction picture, full RWA. Only the Q–E iSWAP coupling is active during evolution; H_{QA} is commented out because the Bell state is prepared by a separate gate sequence. Time-independent; minimal numerical cost. Directly reproduces paper results.
1	$H = H_{\mathrm{QE,int}} + H_{\mathrm{detuning}}$, where $H_{\mathrm{detuning}} \propto \Omega^2/\Delta$	Doubly-rotating frame. Leading-order AC Stark dispersive correction included: $\chi_{QE} = \Omega_{QE}^2/\Delta_{QE}$, distributed as $-\chi/4$ on Q and $+\chi/4$ on E. Bridge toward lab-frame gate design.

3.3. Operator Embedding and Hamiltonians

Three embedding functions project single-qubit operators into the 8-dimensional space $\mathcal{H}_Q \otimes \mathcal{H}_A \otimes \mathcal{H}_E$: `Q(op) = tensor(op,I,I)`, and analogously for `A` and `E`. The two iSWAP coupling Hamiltonians are defined:

$$H_{QA} = \frac{\Omega_{QA}}{2}\big(\sigma_+^Q\sigma_-^A + \sigma_-^Q\sigma_+^A\big), \tag{9}$$

$$H_{QE} = \frac{\Omega_{QE}}{2}\big(\sigma_+^Q\sigma_-^E + \sigma_-^Q\sigma_+^E\big). \tag{10}$$

In the current implementation, `H_interaction()` returns **only** H_{QE} — the Q–A exchange term is commented out, because H_{QA} is used exclusively during Bell-state preparation (a separate gate-sequence step), not during the open-system evolution. `H_detuning` adds the dispersive correction $\delta_Q\sigma_z^Q/2 + \delta_A\sigma_z^A/2 + \delta_E\sigma_z^E/2$ with $\delta_Q = -(\chi_{QA} + \chi_{QE})/4$, $\delta_A = +\chi_{QA}/4$, $\delta_E = +\chi_{QE}/4$.

3.4. Collapse Operators

`build_c_ops(params, gamma_E)` assembles six operators for the full 3-qubit pipeline (Exp. 2–4), appending a seventh when $\gamma > 0$:

$$\mathbf{c} = \Big\{\sqrt{\tfrac{1}{2T_2^{*(Q)}}}\,Q(\sigma_z),\ \sqrt{\tfrac{1}{2T_2^{*(A)}}}\,A(\sigma_z),\ \sqrt{\tfrac{1}{2T_2^{*(E)}}}\,E(\sigma_z),\ \sqrt{\tfrac{1}{T_1^{(Q)}}}\,Q(\sigma_-),\ \sqrt{\tfrac{1}{T_1^{(A)}}}\,A(\sigma_-),\ \sqrt{\tfrac{1}{T_1^{(E)}}}\,E(\sigma_-),\ \sqrt{\gamma}\,E(\sigma_z)\Big\}, \tag{11}$$

where the $\sqrt{\gamma}\,E(\sigma_z)$ term is appended only when $\gamma > 0$. **Note:** Experiment 1 uses its own 2-qubit collapse operators ($\sqrt{1/(2T_2^{*(Q)})}\,\sigma_z^Q$ and $\sqrt{1/(2T_2^{*(A)})}\,\sigma_z^A$) with T1 relaxation commented out, and does not call `build_c_ops`.

3.5. Bell-State Preparation

At level 0, `prepare_initial_state()` constructs the analytic Bell state $|\Psi^+\rangle_{QA}\otimes|0\rangle_E$ directly. At level 1, it simulates the experimental $\sqrt{\mathrm{iSWAP}}$ gate: evolve $|10,0\rangle_{QAE}$ under H_{QA} for $t = \pi/(4\Omega_{QA}) \approx 0.53\,\mu$s,

yielding $(|10\rangle - i|01\rangle)/\sqrt{2}$ with $\mathcal{C} = 1$.

4. Experiment Replication

4.1. Experiment 1 – Markovian Baseline

`exp1()` constructs a 2-qubit (Q–A only) Lindblad simulation, excluding E entirely. It uses only pure-dephasing collapse operators $\{\sqrt{1/(2T_2^{*(Q)})}\,\sigma_z^Q,\ \sqrt{1/(2T_2^{*(A)})}\,\sigma_z^A\}$ (T1 relaxation terms are commented out in this baseline to isolate the pure-dephasing prediction). The time axis is trimmed to start at the first downward crossing of $\mathcal{C} = 0.8$ to match the paper's presentation convention. The numerical result is compared with the analytic prediction:

$$\mathcal{C}(t) = \mathcal{C}_0 \exp\left(-\frac{t}{T_2^{*(Q)}} - \frac{t}{T_2^{*(A)}}\right). \tag{12}$$

Result: $\mathcal{N} = 0.0000$ and $\Gamma_0 = 0.0500\,\mu\text{s}^{-1}$ – both matching the paper exactly. The mesolve curve overlaps the analytic prediction to machine precision, validating all collapse operators and the Bell-state preparation.

4.2. Experiment 2 – Non-Markovian Dynamics

`exp2()` calls `evolve_lindblad(params, gamma_E=0)`. The Q–E iSWAP coupling drives information from Q into E, collapsing Q–A concurrence, then returns it, producing revivals at period $2\pi/\Omega_{QE} \approx 2\,\mu$s. The time axis is trimmed starting from the *third* downward crossing of $\mathcal{C} = 0.8$ (via `_trim_to_c_start`, `nth=3`), matching the paper's convention of beginning measurement after the initial Bell-state preparation transient.

Table 5: Experiment 2 summary: non-Markovianity and revival period.

Quantity	Paper	Sim (level 0)	Sim (level 1)
$\mathcal{N}$	1.4	~ 3.6	~ 2.0
Revival period	$\sim$2 μs	$\sim$2 μs (correct)	$\sim$2 μs (correct)

The overestimation of $\mathcal{N}$ (3.6 vs 1.4 at level 0) reflects that the simulation includes all T1 and T2 dephasing channels simultaneously on all three qubits – channels that slightly reduce the revival amplitude relative to the paper's calibrated conditions. The period of $\approx$2 μs is correctly reproduced, confirming the coupling rate Ω_{QE} implementation.

4.3. Experiment 3 – NM to Markovian Transition

`exp3()` scans $\gamma \in [0.0,\ 10.0]\,\mu\text{s}^{-1}$ across eight values and computes $\mathcal{N}(\gamma)$ at each point. The simulation correctly reproduces:

- $\mathcal{N} > 0$ at $\gamma = 0$; revivals clearly visible.
- Continuous, monotonic suppression of $\mathcal{N}$ with increasing γ.
- $\mathcal{N} \approx 0$ near $\gamma \simeq 1\,\mu\text{s}^{-1}$.
- Fully Markovian (exponential $\mathcal{C}$ decay) at $\gamma \gtrsim 3\,\mu\text{s}^{-1}$.

Both approximation levels show identical qualitative transition; level 1 produces a slightly smoother suppression due to the AC Stark correction.

4.4. Experiment 4 – Quantum Zeno Stabilisation

`exp4()` scans a dense array of 46 γ values from 0.1 to $45.5\,\mu\text{s}^{-1}$. `fit_decay_rate()` extracts Γ_c by log-linear regression on the post-peak concurrence envelope at each γ. `gamma_c_zeno()` evaluates the analytic prediction $\Omega_{QE}^2/(4\gamma) + \Gamma_0$. The simulation reproduces the characteristic $1/\gamma$ dependence and correctly asymptotes to $\Gamma_0 = 0.0500\,\mu\text{s}^{-1}$ at large γ, with the fitted $A/\gamma + B$ curve lying within $\sim 15\%$ of the theoretical Zeno scaling across the Markovian regime.

5. Creative Extensions

Six substantive extensions go beyond the four paper experiments.

5.1. Monte Carlo Quantum Trajectories

`evolve_mcsolve(params, gamma_E, ntraj=200)` implements a Monte Carlo quantum trajectory solver as an alternative to full density-matrix evolution. Each trajectory models a single experimental run: at each step, a quantum jump (dephasing or relaxation) may occur stochastically, with probability

$\Delta t \sum_k \langle L_k^\dagger L_k \rangle$. Averaging over N_{traj} trajectories converges to the density matrix. The Q–A density matrix is reconstructed from 9 two-qubit Pauli `e_ops` to avoid the QuTiP 5 raw-numpy-array storage issue. The interactive notebook (Section 12 → Sec. 10) provides a direct mesolve vs mcsolve concurrence comparison.

5.2. Bloch–Redfield Structured-Bath Modelling

`evolve_born_markov(params, spectral_density)` uses QuTiP's `brmesolve` to go beyond the flat-spectrum GKSL assumption. Real transmons are dominated by $1/f$ flux noise with a strongly frequency-dependent spectral density. The default implementation uses an Ohmic bath:

$$S(\omega) = \alpha|\omega|e^{-|\omega|/\omega_c}\, n_{\text{th}}(\omega), \tag{13}$$

calibrated so the low-frequency limit matches $T_2^{*(Q)}$ at 20 mK. Users can supply a custom $S(\omega)$ (e.g. a Lorentzian centred on the resonator frequency to model photon-shot-noise dephasing).

5.3. Hardware-Faithful Quantum State Tomography

`measure_state(rho_QA, basis, n_shots)` mimics the real experiment: each shot independently samples the Born-rule probability distribution, with correct basis-rotation unitaries ($R_x = H$, $R_y = (|{+y}\rangle \to |0\rangle)$). `full_tomography_shots()` measures all 9 Pauli-basis pairs and saves raw outcome data to `.txt` files. `qst_log_likelihood(df_tomo)` performs MLE over the Cholesky parametrisation $\rho = T^\dagger T/\text{Tr}(T^\dagger T)$ (guaranteeing positive semidefiniteness), matching the James *et al.* approach cited by the paper. The interactive notebook demonstrates shot-count scaling: at $n_{\text{shots}} = 100$ sampling noise is visible; convergence to the mesolve ground truth is achieved at $n_{\text{shots}} \gtrsim 1000$.

5.4. Quantum Process Tomography

`qpt_chi_matrix()` prepares a complete set of input states (Pauli eigenstates of Q–A), evolves each through the open-system channel, performs QST on each output, and reconstructs the process matrix χ:

$$\mathcal{E}(\rho) = \sum_{mn} \chi_{mn} P_m\, \rho\, P_n^\dagger, \tag{14}$$

where $P_k \in \{I, \sigma_x, \sigma_y, \sigma_z\}/\sqrt{2}$. This characterises the complete quantum channel rather than its action on a specific initial state. The $\text{Re}[\chi]$ matrix is displayed as a heatmap for comparison across γ values.

5.5. Custom PennyLane Circuit Runner

`run_custom_circuit(circuit_fn, gamma_E)` bridges the circuit-based and density-matrix paradigms: (i) the user's PennyLane circuit is executed on `default.qubit`; (ii) the resulting state vector is converted to a 3-qubit QuTiP density matrix by tensoring with $|0\rangle_E$; (iii) the state is evolved under the Lindblad master equation; (iv) MLE QST is performed at $t = 0$ to characterise the prepared state. This enables studies of, e.g. W states or GHZ states as initial conditions, and how their entanglement evolves under Q–E coupling.

5.6. Parametric Modulation Module (`modulation.py`)

This standalone module implements the physical coupling mechanism in the lab frame, demonstrating the Jacobi–Anger sideband selection:

- **QA coupling**: Q flux modulated at $\omega_m = \Delta_{QA}/2$; the $n = -2$ Bessel term $g \cdot J_2(\varepsilon/\omega_m)$ is resonant. Effective coupling calibrated at the J_2 maximum: $\beta_{QA} = 3.054$, $J_2(\beta_{QA}) \approx 0.487$.
- **QE coupling**: Q and E driven *anti-phase* at $\omega_m = \Delta_{QE}/4$; differential modulation 2ε gives $g \cdot J_4(2\varepsilon/\omega_m)$ at the $n = -4$ sideband. Anti-phase is essential: co-phase driving cancels the differential frequency modulation identically, producing zero coupling regardless of amplitude.

`scan_amplitude()` sweeps ε over a physically calibrated range and plots peak population transfer vs amplitude, enabling direct identification of the optimal operating point $g_{\text{bare}} \cdot J_n(\beta_{\text{opt}}) = \Omega_{\text{paper}}$.

6. AI Reflection Piece

6.1. Division of Labour

6.2. Successes of AI-Assisted Structuring

The most valuable AI contribution was the module architecture. A flat monolithic script would have entangled parameters, Hamiltonians, solvers, and experiments, making it difficult to add the second approximation level without touching the experiment functions. The AI's five-layer chain – with `H_for_simulation` as a master selector and `DEFAULT_PARAMS` as the single source of truth – made this straightforward.

Table 6: AI versus student task division in this project.

Category	Student (self-written)	AI agent (Claude)
Physics functions	All core functions: operator embedding, Hamiltonians, collapse operators, concurrence, $\mathcal{N}$ measure, Bell-state prep, frame transforms.	None – the AI was not asked to write physics functions.
Module architecture	Selection of five-file structure, dependency rules, param dict.	AI proposed the layered architecture, suggested the `aprox_deg` flag pattern, and scaffolded the module docstrings.
Code explanations	Physics-level verification of all comments against derivations.	AI wrote all inline comments and function docstrings, including the Jacobi–Anger derivation notes in `_bessel_scale()`.
Debugging	All physics bugs (wrong coupling term, missing E term).	Identified three `modulation.py` bugs (in-phase QE cancellation, Bessel amplitude scale, missing anti-phase); flagged QuTiP 5 API changes (`psi0`→`state`, numpy row-vector trajectory storage).

The AI also correctly identified the Bessel-scale calibration logic in `_bessel_scale()`: that the modulation amplitude must be chosen at the Bessel maximum ($\beta_{QA} = 3.054$ for J_2) and the bare coupling must be rescaled as $g_{\text{bare}} = \Omega_{\text{paper}}/J_n(\beta_{\text{opt}})$. This connects the lab-frame parametric modulation physics directly to the interaction-picture coupling constants in the paper.

6.3. Failures and Limitations

Frame assumption not flagged proactively. The AI initially used `H_lab_parametric` (lab-frame Hamiltonian) for `aprox_deg`= 1 without detecting that $\Delta_{QA}/g_{\text{bare}} \approx 0.36 \ll 1$ – inverting the RWA hierarchy. This produced $\mathcal{N} \approx 15.94$ in the dashboard (versus the expected $\sim$2), a clearly unphysical result. The issue was identified only after the student noted the anomaly and queried the frame convention; the AI then correctly diagnosed the hierarchy inversion and proposed the fix (use `H_for_simulation` at both levels).

QuTiP 5 API changes not anticipated. The initial `mcsolve` implementation used the QuTiP 4 keyword `psi0=` (renamed to `state=` in QuTiP 5) and assumed trajectory states were stored as column-vector `Qobj` instances, whereas QuTiP 5 stores them as $(1, 2^n)$ numpy row arrays. These failures were corrected only after runtime errors occurred, not proactively.

6.4. Assessment of Academic Integrity

The use of AI remained within the boundaries of genuine learning. All physics functions were derived and implemented by the student based on understanding of the GKSL equation, the Wootters formula, the Nakajima–Zwanzig memory kernel, and the paper's experimental protocol. The AI served as an accelerator for scaffolding, documentation, and API-level debugging – analogous to a knowledgeable colleague rather than a substitute for physical understanding. The project's most educationally significant outcomes – diagnosing the RWA hierarchy inversion, tracing the rotating-frame assumption to the paper's parametric modulation mechanism, identifying the in-phase QE cancellation bug – all required active physical reasoning by the student that the AI could support but not replace.

7. Summary

This project numerically reproduces all four experiments of Gaikwad *et al.* using a modular Python platform with two approximation levels. Key quantitative agreements are summarised in Table 7.

The six creative extensions – Monte Carlo trajectories, Bloch–Redfield bath, QST, QPT, custom PennyLane circuit runner, and the parametric modulation module – substantially increase the platform's scope beyond the paper, providing a general-purpose open quantum system simulation environment appropriate for further research and educational use.

Appendix A. Code Pipeline and Experiment Replication Functions

This appendix lists every function that constitutes the core simulation pipeline and the four experiment replication protocols. Functions are grouped by module in dependency order.

Table 7: Summary of numerical results vs paper.

Exp.	Observable	Paper	Sim (L0)	Match
1	$\mathcal{N}$	0	0.0000	Exact
1	Γ_0 (μs^{-1})	0.0500	0.0500	Exact
2	$\mathcal{N}$	1.4	~ 3.6	Qualitative
2	Revival period	$\sim 2\,\mu$s	$\sim 2\,\mu$s	Correct
3	$\mathcal{N}(\gamma{=}0) > 0$	Yes	Yes	Correct
3	$\mathcal{N} \to 0$ at $\gamma \simeq 1$	Yes	$\gamma \approx 1.0$	Correct
4	$\Gamma_c \propto 1/\gamma + \Gamma_0$	Yes	Yes ($\pm$15%)	Correct
4	Asymptote Γ_0	0.0500	0.0500	Correct

Appendix A.1. `state_prep.py` – Operators, Hamiltonians, States

Appendix A.1.1. Operator embedding

Listing 1: Three embedding functions that place single-qubit operators into the 8-dimensional Q⊗A⊗E space.

```
I, sx, sy, sz = qt.qeye(2), qt.sigmax(), qt.sigmay(), qt.sigmaz()
s_plus  = qt.sigmap()   # raising  |0><1|
s_minus = qt.sigmam()   # lowering |1><0|

def Q(op): return qt.tensor(op, I, I)   # qubit Q (index 0)
def A(op): return qt.tensor(I, op, I)   # ancilla A (index 1)
def E(op): return qt.tensor(I, I, op)   # environment E (index 2)
```

Appendix A.1.2. Hamiltonian hierarchy

Listing 2: Coupling Hamiltonians and the master selector. `H_interaction` returns **only** `H_QE_int`; the `H_QA_int` term is commented out in the source — the Bell state is prepared by a separate gate step, not included in the evolution Hamiltonian.

```
def H_QA_int(params):
    Om = params.get("Om_QA", DEFAULT_PARAMS["Om_QA"])
    return Om/2 * (qt.tensor(s_plus, s_minus, I)
                 + qt.tensor(s_minus, s_plus, I))

def H_QE_int(params):
    Om = params.get("Om_QE", DEFAULT_PARAMS["Om_QE"])
    return Om/2 * (qt.tensor(s_plus, I, s_minus)
                 + qt.tensor(s_minus, I, s_plus))

def H_interaction(params):
    return H_QE_int(params)   # H_QA_int commented out in current source

def H_detuning(params):
    # Leading-order AC Stark dispersive correction (aprox_deg = 1)
    chi_QA = Om_QA**2 / delta_QA
    chi_QE = Om_QE**2 / delta_QE
    dQ = -(chi_QA + chi_QE) / 4.0
    dA = +chi_QA / 4.0
    dE = +chi_QE / 4.0
    return dQ*Q(sz)/2 + dA*A(sz)/2 + dE*E(sz)/2

def H_for_simulation(params):
    deg = params.get("aprox_deg", 0)
    if deg == 0:
        return H_interaction(params)                        # = H_QE_int only
    else:
        return H_interaction(params) + H_detuning(params)  # + AC Stark
```

Appendix A.1.3. Bell-state preparation

Listing 3: Initial state preparation for both approximation levels.

```
def prepare_initial_state(params=None):
    deg = params.get("aprox_deg", 0)
    if deg == 0:
        Psi_plus = qt.bell_state('10')        # (|10>+|01>)/sqrt(2) analytic
        return qt.ket2dm(qt.tensor(Psi_plus, qt.basis(2, 0)))
    else:
        # Simulate sqrt(iSWAP) gate: evolve |10,0>_QAE under H_QA
        psi0   = qt.tensor(sx*qt.basis(2,0), qt.basis(2,0), qt.basis(2,0))
        rho0   = qt.ket2dm(psi0)
        t_gate = np.pi / (4.0 * params.get("Om_QA", DEFAULT_PARAMS["Om_QA"]))
        result = qt.mesolve(H_QA_int(params), rho0, [0.0, t_gate], [], [])
        return result.states[-1]          # (|10>-i|01>)/sqrt(2), C=1
```

Appendix A.1.4. Non-Markovianity measure and Wootters concurrence

Listing 4: RHP non-Markovianity measure (actual implementation) and manual Wootters concurrence. Note: the code computes `integral + (C[-1]-C[0])` which equals $\int |dC/dt|dt - (C_0 - C_f) = \int |dC/dt|dt - \Delta C$, matching the paper's formula exactly since $\Delta C = C(t_0) - C(t_f)$.

```
def nm_measure(conc_list, tlist):
    C  = np.asarray(conc_list, dtype=float)
    t  = np.asarray(tlist,     dtype=float)
    dC = np.gradient(C, t)
    # integral + (C_f - C_0) = integral - (C_0 - C_f) = integral - DeltaC
    return float(trapezoid(np.abs(dC), t) + (C[-1] - C[0]))

def concurrence_manual(rho_qobj):
    rho  = rho_qobj.full()
    sysy = np.kron(np.array([[0,-1j],[1j,0]]), np.array([[0,-1j],[1j,0]]))
    R    = rho @ (sysy @ rho.conj() @ sysy)
    eigs = np.sort(np.maximum(np.real(np.linalg.eigvals(R)), 0))[::-1]
    seig = np.sqrt(eigs)
    return max(0.0, seig[0]-seig[1]-seig[2]-seig[3]), seig
```

Appendix A.2. `dynamical.py` – Collapse Operators and Primary Solver

Appendix A.2.1. Collapse operator builder

Listing 5: Assembles all Lindblad collapse operators for the three-qubit system.

```
def build_c_ops(params, gamma_E=0.0):
    T2Q, T2A, T2E = params["T2Q"], params["T2A"], params["T2E"]
    T1Q, T1A, T1E = params["T1Q"], params["T1A"], params["T1E"]
    ops = [
        np.sqrt(1/(2*T2Q)) * Q(sz),    # Q pure dephasing
        np.sqrt(1/(2*T2A)) * A(sz),    # A pure dephasing
        np.sqrt(1/(2*T2E)) * E(sz),    # E intrinsic dephasing
        np.sqrt(1/T1Q)     * Q(s_minus),  # Q T1 relaxation
        np.sqrt(1/T1A)     * A(s_minus),  # A T1 relaxation
        np.sqrt(1/T1E)     * E(s_minus),  # E T1 relaxation
    ]
    if gamma_E > 0.0:
        ops.append(np.sqrt(gamma_E) * E(sz))  # tunable E dephasing
    return ops
```

Appendix A.2.2. Primary Lindblad solver

Listing 6: GKSL master equation solver; entry point for all four experiments.

```
def evolve_lindblad(params, gamma_E=0.0, tlist=None, rho0=None):
    if tlist is None: tlist = np.linspace(0, 10, 400)
    if rho0  is None: rho0  = prepare_initial_state(params)
    c_ops  = build_c_ops(params, gamma_E)
    H      = H_for_simulation(params)   # selects level 0 or 1
    result = qt.mesolve(H, rho0, tlist, c_ops, e_ops=[],
                        options={'nsteps':8000,'atol':1e-9,'rtol':1e-7})
    conc   = [qt.concurrence(s.ptrace([0,1])) for s in result.states]
    return np.array(tlist), result.states, conc
```

Appendix A.2.3. Zeno scaling utilities

Listing 7: Analytic Zeno prediction and log-linear decay-rate extractor.

```
def gamma_c_zeno(gamma_E, params):
    Om_QE  = params.get("Om_QE", DEFAULT_PARAMS["Om_QE"])
    Gamma0 = 1/params["T2Q"] + 1/params["T2A"]
    return Om_QE**2 / (4.0 * gamma_E) + Gamma0

def fit_decay_rate(conc, tlist, skip_frac=0.05):
    skip  = max(1, int(skip_frac * len(tlist)))
    C     = np.maximum(conc[skip:], 1e-10)
    t     = tlist[skip:]
    valid = np.isfinite(np.log(C)) & (C > 1e-9)
    if valid.sum() < 5: return np.nan
    coeff = np.polyfit(t[valid], np.log(C[valid]), 1)
    return float(-coeff[0])
```

Appendix A.3. `experiment.py` – Experiment Protocols

Appendix A.3.1. `exp1()` – Markovian baseline

Listing 8: Experiment 1: 2-qubit Q-A only simulation. T1 relaxation operators are commented out in the source; only pure-dephasing σ_z operators are used. Time axis trimmed at first downward crossing of C=0.8.

```
def exp1(params=None, tlist=None, plot=True):
    T2Q, T2A = params["T2Q"], params["T2A"]
    Om_QA    = params.get("Om_QA", DEFAULT_PARAMS["Om_QA"])
    # 2-qubit system: Q and A only, E excluded entirely
    H_2q = Om_QA/2 * (qt.tensor(sp2,sm2) + qt.tensor(sm2,sp2))
    # T1 operators commented out in source -- pure dephasing only
    c_Q2 = np.sqrt(1.0/(2*T2Q)) * qt.tensor(sz2, I2)
    c_A2 = np.sqrt(1.0/(2*T2A)) * qt.tensor(I2,  sz2)
    rho0 = qt.ket2dm(qt.bell_state('10'))
```

```
    result = qt.mesolve(H_2q, rho0, tlist, [c_Q2, c_A2], [],
                        options={'nsteps': 5000})
    conc   = [qt.concurrence(s) for s in result.states]
    # Trim at FIRST downward crossing of C=0.8
    below    = np.where(np.asarray(conc) <= 0.8)[0]
    trim_idx = below[0] if len(below) > 0 else 0
    t_trim, conc_trim = tlist[trim_idx:]-tlist[trim_idx], conc[trim_idx:]
    # Expected: NM = 0, decay matches C0*exp(-t/T2Q - t/T2A)
    return {'tlist':t_trim, 'conc':conc_trim,
            'NM':nm_measure(conc_trim, t_trim), 'Gamma0':1/T2Q+1/T2A, ...}
```

Appendix A.3.2. `exp2()` *– Non-Markovian dynamics*

Listing 9: Experiment 2: Q-E coupling active at $\gamma_E = 0$. Time trimmed at 3rd downward crossing of C=0.8 (`_trim_to_c_start`, `nth=3`) to match the paper's presentation window.

```
def exp2(params=None, tlist=None, plot=True, baseline=None):
    # H_for_simulation = H_QE_int only; gamma_E=0 means no L_E collapse op
    t, states, conc = evolve_lindblad(params, gamma_E=0.0, tlist=tlist)
    # Trim at 3rd downward crossing of C=0.8 (nth=3)
    t_trim, conc_trim = _trim_to_c_start(conc, t, c_start=0.8, nth=3)
    NM = nm_measure(conc_trim, t_trim)
    # Expected: NM > 0; revival period ~ 2*pi/Omega_QE ~ 2 us
    return {'tlist':t_trim, 'conc':conc_trim, 'NM':NM, 'states':states, ...}
```

Appendix A.3.3. `exp3()` *– NM to Markovian transition*

Listing 10: Experiment 3: sweeps γ_E and computes $\mathcal{N}(\gamma)$ at each point.

```
def exp3(params=None, tlist=None, gamma_scan=None, plot=True):
    gamma_scan = gamma_scan or [0.0, 0.2, 0.5, 1.0, 2.0, 3.0, 5.0, 10.0]
    all_conc, all_NM = {}, {}
    for g in gamma_scan:
        t, _, conc        = evolve_lindblad(params, gamma_E=g, tlist=tlist)
        t_tr, conc_tr     = _trim_to_c_start(conc, t)
        all_conc[g]       = (t_tr, conc_tr)
        all_NM[g]         = nm_measure(conc_tr, t_tr)
    # Expected: NM(0)>0, NM continuous to 0 near gamma~1, Markovian at gamma~3
    return {'gamma_scan':gamma_scan, 'all_conc':all_conc, 'all_NM':all_NM}
```

Appendix A.3.4. `exp4()` *– Quantum Zeno stabilisation*

Listing 11: Experiment 4: dense γ_E scan at large values; fits Γ_c vs γ.

```
def exp4(params=None, tlist=None, gamma_zeno_scan=None, plot=True):
    Gamma_c_sim, Gamma_c_pred, all_conc = [], [], {}
    for g in gamma_zeno_scan:
        t, _, conc    = evolve_lindblad(params, gamma_E=g, tlist=tlist)
        t_tr, conc_tr = _trim_to_c_start(conc, t)
        Gc_sim        = fit_decay_rate(np.array(conc_tr), t_tr)
        Gc_pred       = gamma_c_zeno(g, params)
        Gamma_c_sim.append(Gc_sim); Gamma_c_pred.append(Gc_pred)
    # Expected: Gamma_c ~ Omega_QE^2/(4*gamma) + Gamma0, asymptote at Gamma0
    return {'gamma_scan':gamma_zeno_scan, 'Gamma_c_sim':Gamma_c_sim, ...}
```

Appendix A.4. `main.py` *– Entry-Point Pipeline*

Listing 12: Top-level execution script: runs all four experiments sequentially and generates the dashboard PDF.

```
PARAMS = DEFAULT_PARAMS.copy()
PARAMS["aprox_deg"] = 0        # interaction picture (exact paper reproduction)
TLIST  = np.linspace(0, 10, 400)
GAMMA_SCAN      = [0.0, 0.2, 0.5, 1.0, 2.0, 3.0, 5.0, 10.0]
GAMMA_ZENO_SCAN = [0.1, 0.2, ..., 45.5]   # 46 values

def main():
    res1 = exp1(params=PARAMS, tlist=TLIST, plot=True)
    res2 = exp2(params=PARAMS, tlist=TLIST, plot=True, baseline=res1['conc'])
    res3 = exp3(params=PARAMS, tlist=TLIST, gamma_scan=GAMMA_SCAN, plot=True)
    res4 = exp4(params=PARAMS, tlist=TLIST, gamma_zeno_scan=GAMMA_ZENO_SCAN,
                plot=True)
    plot_full_dashboard(res1, res2, res3, res4, params=PARAMS, save_pdf=True)
```

Appendix B. Creative Extension Functions

This appendix lists every function that goes beyond the four paper experiments, implemented as creative extensions of the platform.

Appendix B.1. Extension 1 – Monte Carlo Quantum Trajectories

Listing 13: `evolve_mcsolve()`: stochastic quantum trajectory solver using QuTiP 5 API. Each trajectory models one experimental shot.

```
def evolve_mcsolve(params, gamma_E=0.0, tlist=None, rho0=None, ntraj=200):
    # mcsolve requires a pure ket; extract if rho0 is a pure density matrix
```

```
    if rho0.type == 'oper':
        purity = (rho0 * rho0).tr()
        if abs(purity - 1.0) < 1e-6:
            evals, evecs = rho0.eigenstates()
            psi0 = evecs[np.argmax(np.real(evals))]
        else:
            return evolve_lindblad(params, gamma_E, tlist, rho0)  # fallback
    # Use 9 Pauli e_ops to reconstruct rho_QA -- avoids QuTiP 5 numpy storage issue
    paulis = [sx, sy, sz]
    e_ops  = [qt.tensor(p1, p2, qt.qeye(2)) for p1 in paulis for p2 in paulis]
    result = qt.mcsolve(H=H, state=psi0, tlist=tlist,   # state= not psi0= (QuTiP 5)
                        c_ops=c_ops, e_ops=e_ops, ntraj=ntraj,
                        options={'nsteps':8000, 'progress_bar':False})
    conc, states = [], []
    for i in range(len(tlist)):
        rho_QA = _rho_from_pauli_expect(result.expect, i)
        states.append(rho_QA); conc.append(qt.concurrence(rho_QA))
    return np.array(tlist), states, conc

def _rho_from_pauli_expect(expect, t_idx):
    """Reconstruct␣2-qubit␣rho_QA␣from␣9␣Pauli␣expectation␣values."""
    rho = qt.qeye([2,2]) / 4.0
    for k, (p1, p2) in enumerate((p1,p2) for p1 in [sx,sy,sz]
                                          for p2 in [sx,sy,sz]):
        rho = rho + 0.25 * float(np.real(expect[k][t_idx])) * qt.tensor(p1,p2)
    rho.dims = [[2,2],[2,2]]
    return rho
```

Appendix B.2. Extension 2 – Bloch–Redfield Structured-Bath Modelling

Listing 14: `evolve_born_markov()`: Bloch–Redfield equation with user-supplied spectral density $S(\omega)$.

```
def evolve_born_markov(params, spectral_density=None, tlist=None, rho0=None):
    if spectral_density is None:
        # Default: Ohmic bath calibrated to T2Q at 20 mK
        alpha, omega_c, kBT = 1/(2*np.pi*T2Q), 2*np.pi*5.0, 0.04
        def spectral_density(omega):
            if abs(omega) < 1e-12: return alpha * omega_c * kBT
            S = alpha * abs(omega) * np.exp(-abs(omega)/omega_c)
            n = 1/(np.exp(abs(omega)/kBT) - 1 + 1e-12)
            return S*(n+1) if omega > 0 else S*n

    H     = H_for_simulation(params)
    a_ops = [[Q(sz), spectral_density], [A(sz), spectral_density]]
    try:
        result = qt.brmesolve(H=H, psi0=rho0, tlist=tlist,
                              a_ops=a_ops, e_ops=[])
        conc = [qt.concurrence(s.ptrace([0,1])) for s in result.states]
        return np.array(tlist), result.states, conc
    except Exception:
        return evolve_lindblad(params, gamma_E=0.0, tlist=tlist, rho0=rho0)
```

Appendix B.3. Extension 3 – Hardware-Faithful Quantum State Tomography

Listing 15: `measure_state()`: simulates projective measurement by multinomial sampling from the Born-rule distribution with basis rotation.

```
def measure_state(rho_QA, basis='z', n_shots=1024, save_file=None, seed=None):
    rng = np.random.default_rng(seed)
    if isinstance(basis, str): basis = [basis, basis]
    def _to_z(b):  # rotate measurement basis into Z basis
        if b == 'x': return qt.Qobj(np.array([[1,1],[1,-1]])/np.sqrt(2))
        if b == 'y': return qt.Qobj(np.array([[1,-1j],[1,1j]])/np.sqrt(2))
        return qt.qeye(2)
    R = qt.tensor(_to_z(basis[0]), _to_z(basis[1]))
    rho_meas = R * rho_QA * R.dag()
    probs = np.maximum(np.real(np.diag(rho_meas.full())), 0)
    probs /= probs.sum()
    outcome_map = {0:(1,1), 1:(1,-1), 2:(-1,1), 3:(-1,-1)}
    indices  = rng.choice(4, size=n_shots, p=probs)
    outcomes = np.array([outcome_map[i] for i in indices])
    if save_file: # save to measurement_results/*.txt
        ...
    return outcomes, counts

def full_tomography_shots(rho_QA, n_shots=1024, exp_label="exp",
                          t_label="t0", seed=None):
    """Collect␣shots␣in␣all␣9␣Pauli-basis␣combinations;␣saves␣.txt␣files."""
    frames = []
    for bQ in ['x','y','z']:
        for bA in ['x','y','z']:
            s, _ = measure_state(rho_QA, [bQ,bA], n_shots,
                                 save_file=f"{exp_label}_{t_label}_{bQ}{bA}.txt")
            df = pd.DataFrame(s, columns=['q_out','a_out'])
            df['basis_Q'], df['basis_A'] = bQ, bA
            frames.append(df)
    return pd.concat(frames, ignore_index=True)
```

Appendix B.4. Extension 4 – Maximum-Likelihood Quantum State Tomography

Listing 16: `qst_log_likelihood()`: MLE QST from shot DataFrame; matches HW2.ipynb approach.

```
def qst_log_likelihood(df_tomo):
    def _mle_single(df, out_col, basis_col):
        """MLE␣Bloch␣vector␣for␣one␣qubit␣via␣Nelder-Mead␣minimisation."""
        counts = {}
        for b in ['x','y','z']:
            sub = df[df[basis_col]==b][out_col].values
            Np, Nm = int((sub==1).sum()), int((sub==-1).sum())
            counts[b] = (Np, Nm)
        def neg_log_like(p):
            rx,ry,rz = p
            if rx**2+ry**2+rz**2 > 1.0: return 1e10
            val = 0.0
            for b,r in [('x',rx),('y',ry),('z',rz)]:
                Np,Nm = counts[b]
                val -= Np*np.log(max((1+r)/2, 1e-15))
                val -= Nm*np.log(max((1-r)/2, 1e-15))
            return val
        return opt.minimize(neg_log_like, [0,0,0], method='Nelder-Mead').x

    bloch_Q = _mle_single(df_tomo, 'q_out', 'basis_Q')
    bloch_A = _mle_single(df_tomo, 'a_out', 'basis_A')
    # Build 2-qubit rho from marginals + two-qubit Pauli correlators
    rho_QA = qt.qeye([2,2]) / 4.0
    for bQ in ['x','y','z']:
        for bA in ['x','y','z']:
            sub  = df_tomo[(df_tomo['basis_Q']==bQ)&(df_tomo['basis_A']==bA)]
            corr = float((sub['q_out'].values*sub['a_out'].values).mean())
            rho_QA += 0.25 * corr * qt.tensor(pauli_map[bQ], pauli_map[bA])
    # Add single-qubit marginal contributions
    rx_Q,ry_Q,rz_Q = bloch_Q; rx_A,ry_A,rz_A = bloch_A
    rho_QA += (0.25*rx_Q*qt.tensor(sx,I2) + 0.25*ry_Q*qt.tensor(sy,I2)
              + 0.25*rz_Q*qt.tensor(sz,I2) + 0.25*rx_A*qt.tensor(I2,sx)
              + 0.25*ry_A*qt.tensor(I2,sy) + 0.25*rz_A*qt.tensor(I2,sz))
    rho_QA.dims = [[2,2],[2,2]]
    return rho_QA, bloch_Q, bloch_A
```

Appendix B.5. Extension 5 – Quantum Process Tomography

Listing 17: `qpt_chi_matrix()`: reconstructs the process matrix χ for a user-supplied channel.

```
def qpt_chi_matrix(channel_fn, n_input_states=4):
    # Input states: |0>, |1>, |+>, |+i>
    rho_basis = [qt.ket2dm(qt.basis(2,0)), qt.ket2dm(qt.basis(2,1)),
                 qt.ket2dm((qt.basis(2,0)+qt.basis(2,1)).unit()),
                 qt.ket2dm((qt.basis(2,0)+1j*qt.basis(2,1)).unit())]
    # Operator basis {I, X, iY, Z}
    E_ops = [qt.qeye(2), qt.sigmax(),
             qt.Qobj([[0,-1j],[1j,0]]), qt.sigmaz()]
    n = len(E_ops)
    Lambda_vec = np.zeros(4*n, dtype=complex)
    Beta       = np.zeros((4*n, n*n), dtype=complex)
    for j, rho_j in enumerate(rho_basis):
        rho_out   = channel_fn(rho_j)
        rho_out_v = rho_out.full().reshape(4)
        for k in range(4):
            Lambda_vec[j*4+k] = rho_out_v[k]
            for m in range(n):
                for p in range(n):
                    E_m_rho_Ep = (E_ops[m]*rho_j*E_ops[p].dag()).full().reshape(4)
                    Beta[j*4+k, m*n+p] = E_m_rho_Ep[k]
    chi_vec = np.linalg.lstsq(Beta, Lambda_vec, rcond=None)[0]
    return np.real(chi_vec.reshape(n, n))
```

Appendix B.6. Extension 6 – Custom PennyLane Circuit Runner

Listing 18: `run_custom_circuit()`: executes any PennyLane circuit as the initial state and evolves it through the open-system pipeline.

```
def run_custom_circuit(circuit_fn, gamma_E=0.0, params=None,
                       tlist=None, n_shots=1024, plot=True, label="Custom"):
    # Step 1: execute PennyLane circuit, get state vector
    dev   = qml.device('default.qubit', wires=2)
    qnode = qml.QNode(circuit_fn, dev)
    sv    = np.array(qnode())
    rho_QA = qt.Qobj(np.outer(sv, sv.conj()), dims=[[2,2],[2,2]])
    print(f"[{label}]␣␣Initial␣C0␣=␣{qt.concurrence(rho_QA):.4f}")

    # Step 2: embed into 3-qubit space (E = ground state)
    rho0_3q = qt.tensor(rho_QA, qt.ket2dm(qt.basis(2,0)))

    # Step 3: Lindblad evolution
    t, states, conc = evolve_lindblad(params, gamma_E=gamma_E,
                                      tlist=tlist, rho0=rho0_3q)
    NM = nm_measure(conc, t)
```

```
    # Step 4: MLE QST on initial state (n_shots shots)
    df_tomo          = full_tomography_shots(rho_QA, n_shots=n_shots)
    rho_mle, _, _    = qst_log_likelihood(df_tomo)

    return {'tlist':t,'conc':conc,'NM':NM,'states':states,
            'initial_rho_QA':rho_QA, 'qst_rho':rho_mle}

# Example usage:
# def bell_circuit():
#     qml.Hadamard(wires=0); qml.CNOT(wires=[0,1]); return qml.state()
# result = run_custom_circuit(bell_circuit, gamma_E=1.0)
```

Appendix B.7. Extension 7 – Parametric Modulation Module

Listing 19: `modulation.py`: lab-frame Hamiltonian for the Jacobi–Anger parametric coupling demonstration.

```
def H_modulation_lab(params=DEFAULT_PARAMS, mod_type="QA",
                     epsilon=0.0, wm=None):
    detuning = om_Q - om_A if mod_type=="QA" else om_Q - om_E
    if   mod_type=="QA" and wm is None: wm = abs(detuning)/2  # drive at Delta/2
    elif mod_type=="QE" and wm is None: wm = abs(detuning)/4  # drive at Delta/4
    H0 = H_free(params)
    if mod_type == "QA":
        H_static = H0 + H_QA_int(params)
        H_mod_op = Q(sz) / 2.0              # only Q has FFL
    else:
        H_static = H0 + H_QE_int(params)
        H_mod_op = Q(sz)/2.0 - E(sz)/2.0   # anti-phase: doubles beta'=2eps/wm
    def _coeff(t, args):
        return epsilon * np.cos(args["wm"] * t)
    return [H_static, [H_mod_op, _coeff]], wm

def scan_amplitude(params=DEFAULT_PARAMS, mod_type="QA", ...):
    """Sweep␣epsilon␣over␣physically␣calibrated␣range;␣record␣max␣population
␣␣␣␣transfer␣and␣estimated␣swap␣rate␣at␣each␣amplitude."""
    if mod_type == "QA":
        eps_list = np.linspace(0, 0.5*abs(om_Q-om_A), 30)  # up to 0.5*Delta_QA
    else:
        eps_list = np.linspace(0, 1.0*abs(om_Q-om_E), 30)  # up to Delta_QE
    for eps in eps_list:
        result, _ = run_modulation(params, mod_type, epsilon=eps, ...)
        max_transfer.append(np.max(result.expect[1]))
        # estimate swap rate from peak spacing
    return eps_list, max_transfer, swap_rates, wm_used
```

References

[1] Gaikwad, C., Kowsari, D., Brame, C., Song, X., Zhang, H., Esposito, M., Ranadive, A., Cappelli, G., Roch, N., Levenson-Falk, E. M., & Murch, K. W. (2024). Entanglement assisted probe of the non-Markovian to Markovian transition in open quantum system dynamics. *Physical Review Letters*, **132**, 200401.

[2] Breuer, H.-P., & Petruccione, F. (2002). *The Theory of Open Quantum Systems*. Oxford University Press.

[3] Rivas, Á., Huelga, S. F., & Plenio, M. B. (2010). Entanglement and non-Markovianity of quantum evolutions. *Physical Review Letters*, **105**, 050403.

[4] Wootters, W. K. (1998). Entanglement of formation of an arbitrary state of two qubits. *Physical Review Letters*, **80**, 2245.

[5] Nielsen, M. A., & Chuang, I. L. (2000). *Quantum Computation and Quantum Information*. Cambridge University Press.

[6] Johansson, J. R., Nation, P. D., & Nori, F. (2012). QuTiP: An open-source Python framework for the dynamics of open quantum systems. *Computer Physics Communications*, **183**, 1760.

[7] James, D. F. V., Kwiat, P. G., Munro, W. J., & White, A. G. (2001). Measurement of qubits. *Physical Review A*, **64**, 052312.